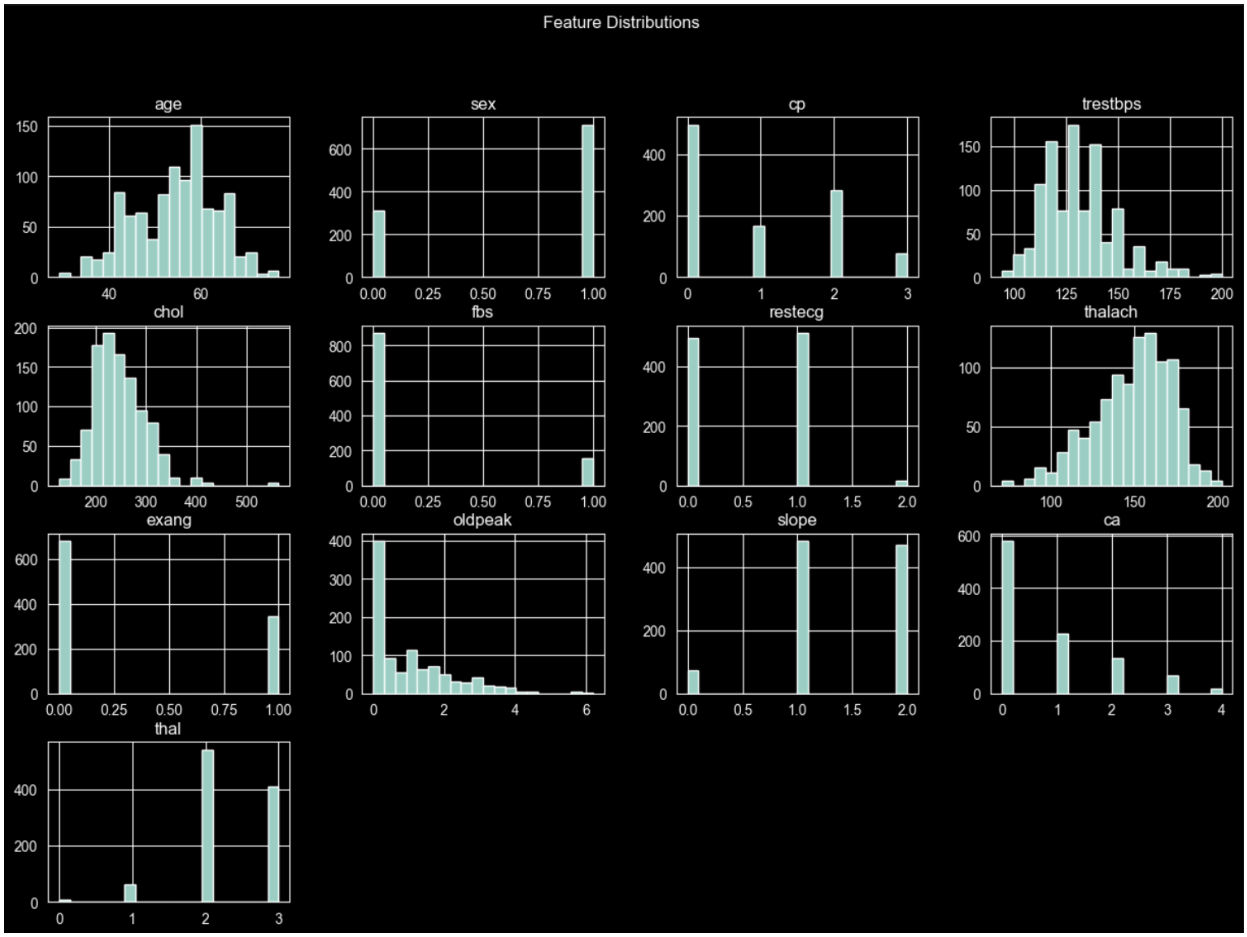
Data & Preprocessing

Inspect the datasets (shapes, missing values, feature distributions, class balance):

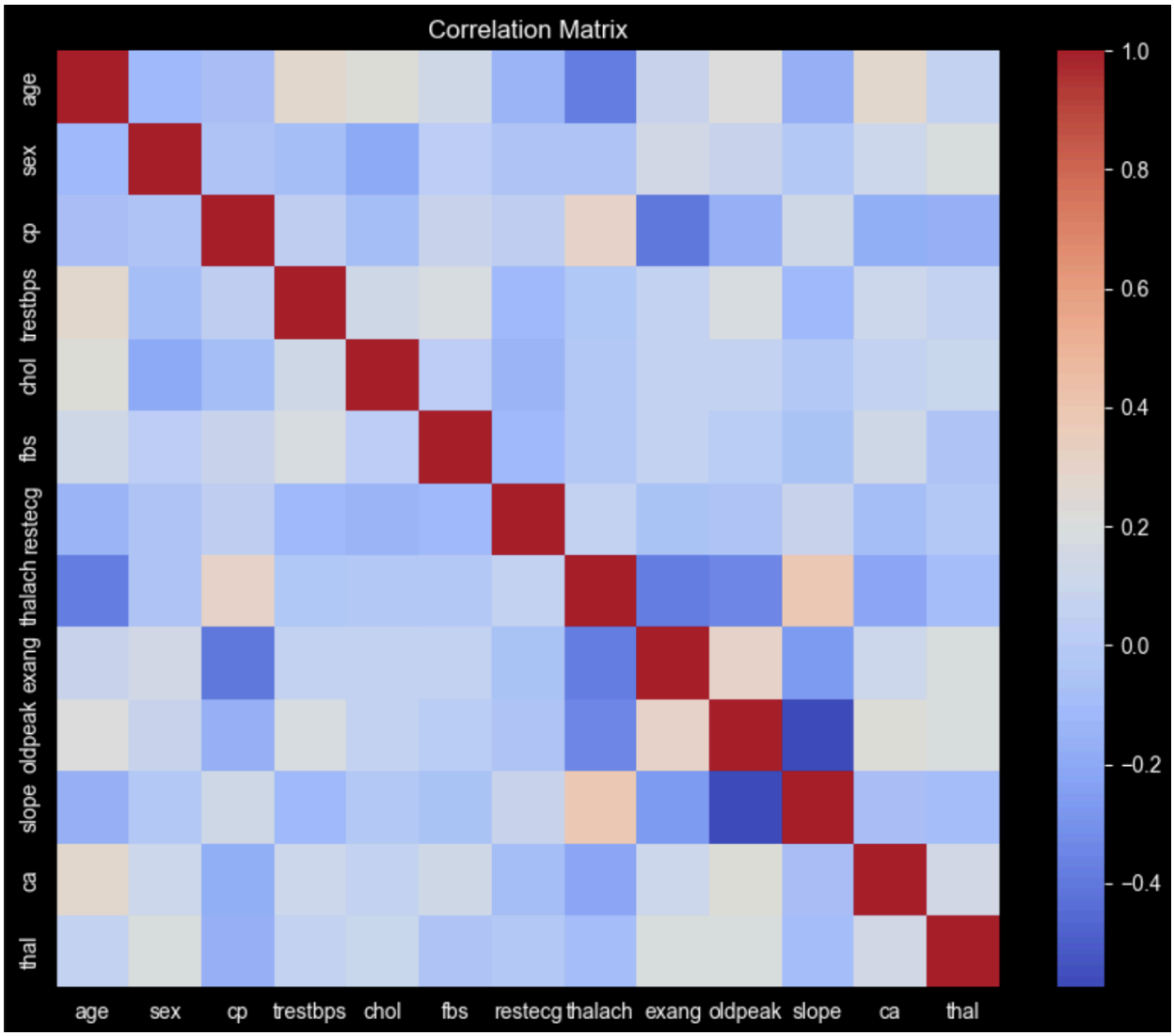
Heart:

Similar to the previous homeworks, there were no missing values in our heart disease dataset. The data has already been preprocessed and prefiltered for initial feature selection based on variance.

The dataset consists of 13 features and 1025 samples. Below is the distribution of each feature. We used the standard scaler for this dataset, which normalizes to a mean of 0 and standard deviation of 1. PCA was tested with 2 components, 10 components, and finally all 13. Below are the feature distributions, correlation matrix, class balance, and confusion matrices for the model with PCA 2, 10, and no PCA respectively.



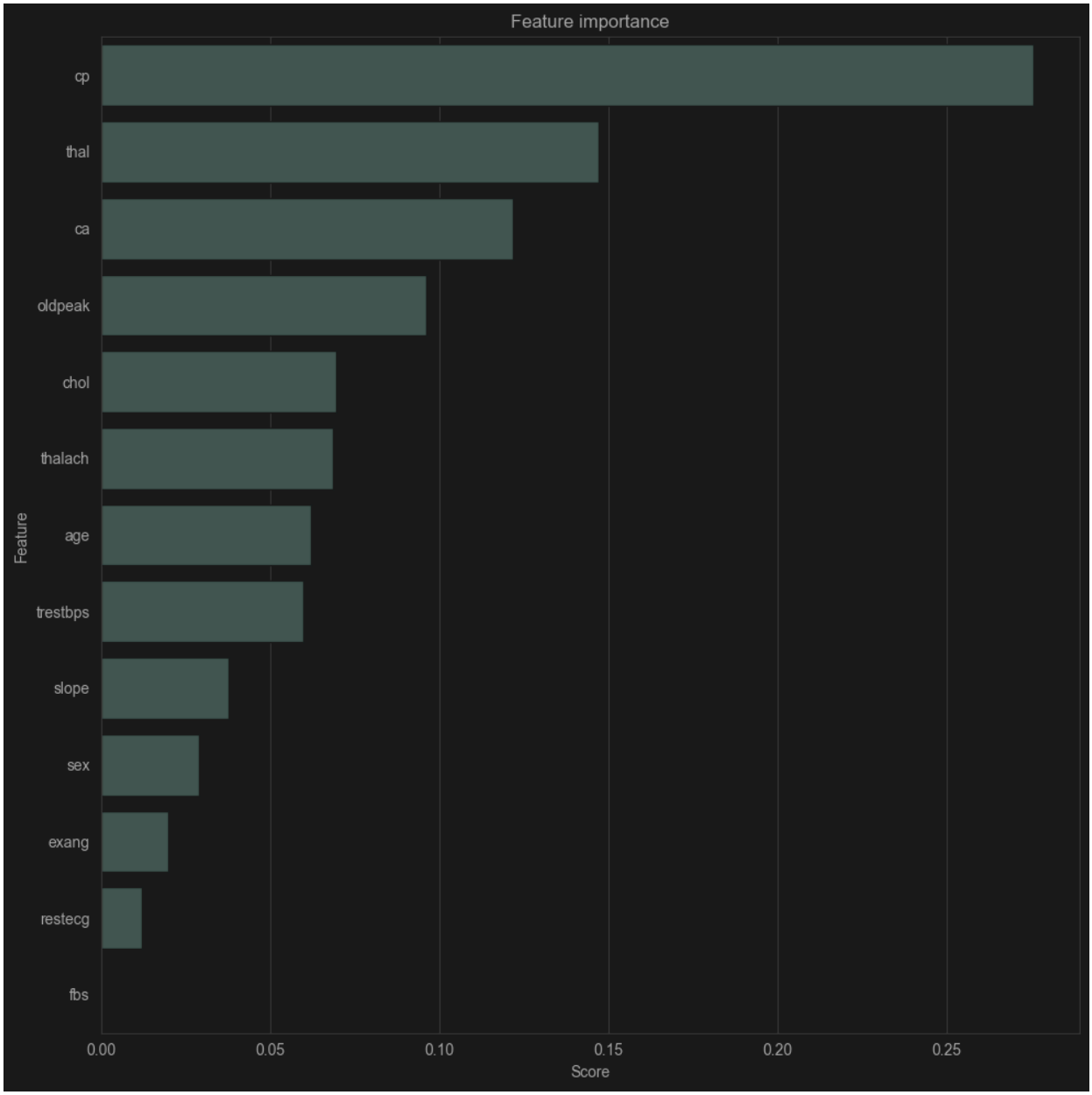
As we can see here, there is an outlier in oldpeak and chol, but the rest of numeric features have reasonable ranges and standard deviations (omitting the table of all numbers for space, but it was on the last two homeworks also).



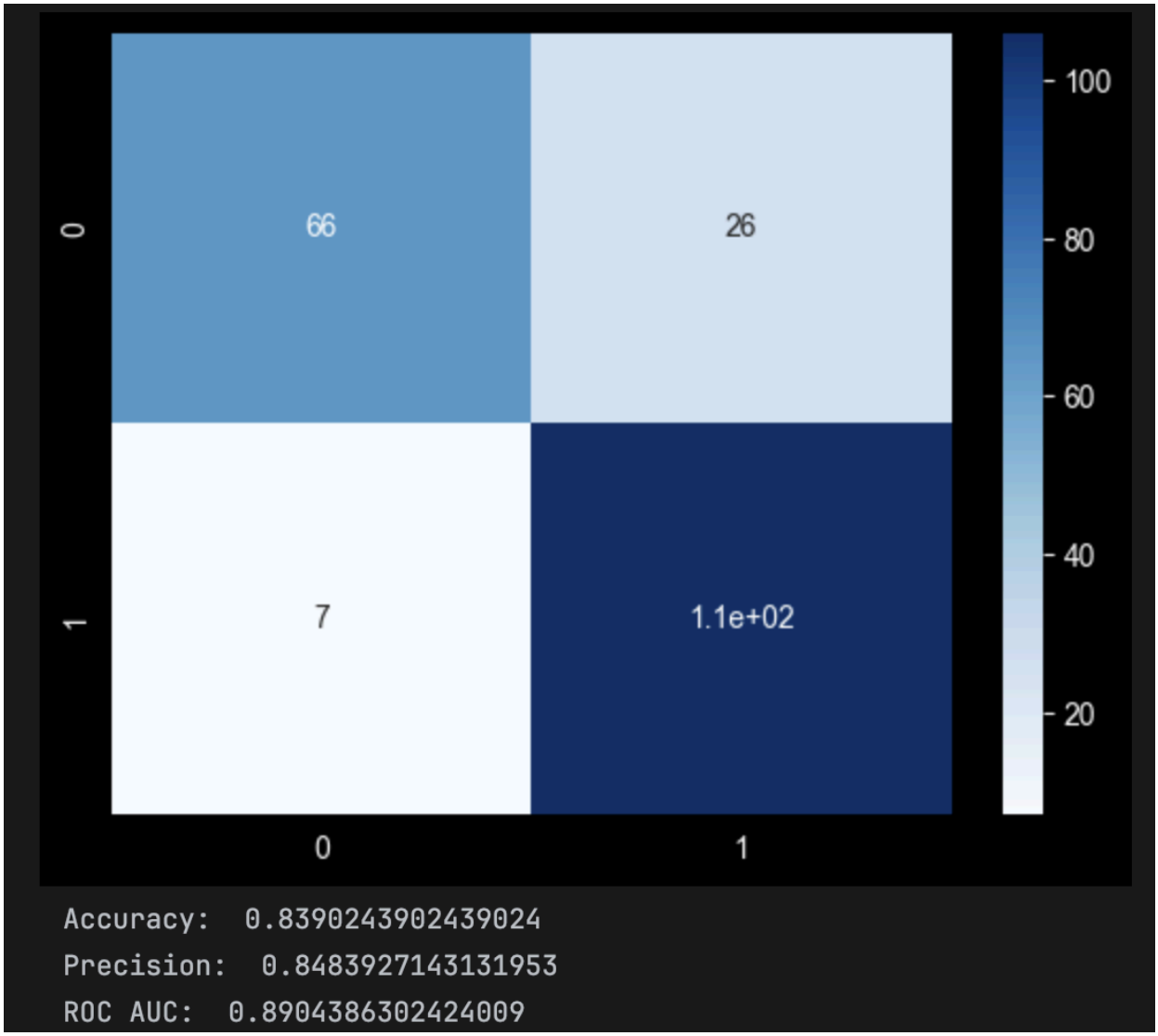
We see in our correlation matrix that oldpeak and slope are negatively correlated with each other. The value is not too high, but it could be worthwhile to investigate whether we can remove one of the features if the dataset was much larger.

The two classes are almost perfectly balanced. There should be no need to worry about whether either class will be overrepresented or not.

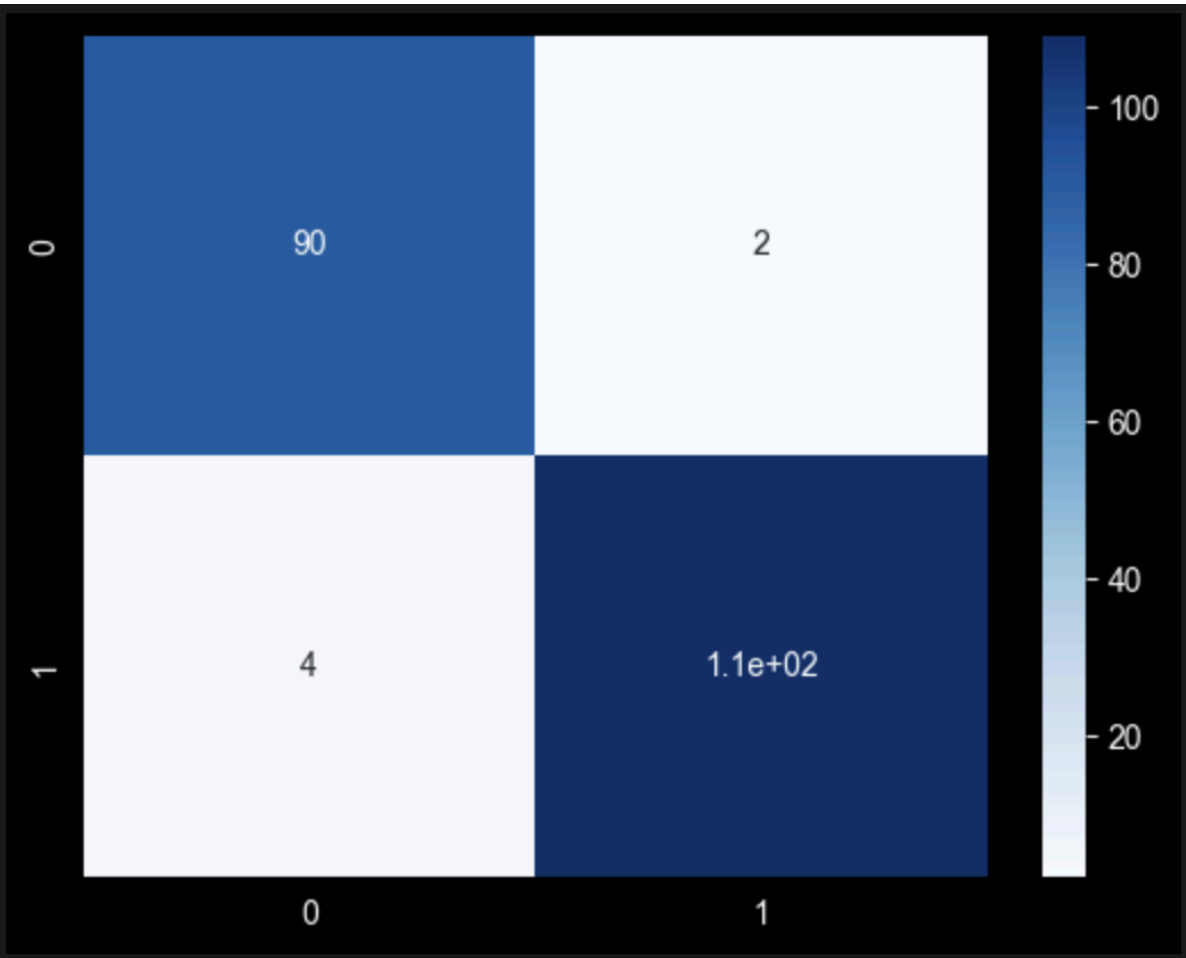
Here it should be clear that all features except fbs should have a role in classification but cp, thal, and ca contribute the most.



PCA 2 = cp, thal. Worked reasonably well for only including two features.

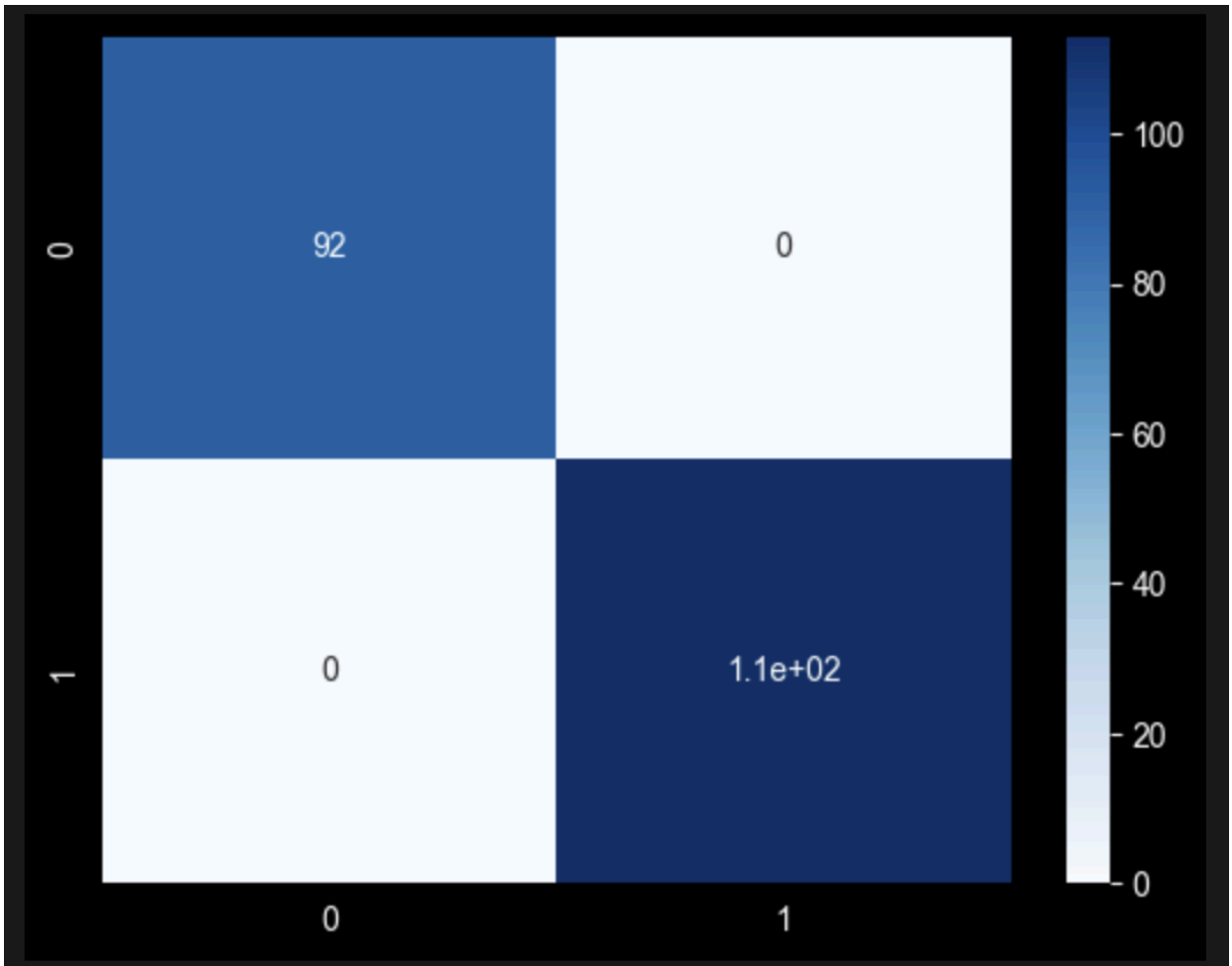


PCA = 10 (all features except: exang, restecg, fbs). This worked very well but was not as good as using all features. Shows that potentially all features are relevant for the model. With PCA: 12, the model was able to get to 99% accuracy.




```
Accuracy:  0.9707317073170731
Precision:  0.9709710748631352
ROC AUC:  0.9848980377068104
```

With all features present, the model is able to classify correctly 100% of the time.




```
Accuracy:  1.0
Precision:  1.0
ROC AUC:  1.0
```

Aging:

For the aging dataset, I had to first drop the non numeric columns (dataset, tissue_type, and gender) and then drop “age” since that was our target column. There were missing values for the aging dataset, and I did have to drop one feature column that contained no actual values in it, which was preventing my mean imputation. The dataset consisted of 37 samples and 485515 features. My prediction was that this massive discrepancy would lead to overfitting of the data and poor performance, which the results seem to back up. 265 features were missing values and we will be dealing with this through mean imputation in our pipeline to prevent leakage. I attempted PCA with 20 components and then all features, and the pca results were worse but not by a large margin, which leads me to believe that there is quite a lot of noise in the dataset and many of the features are redundant.

Build leakage-free Pipelines that include: imputation → scaling → (optional PCA) → SVM/SVR

Heart:

I ran two pipelines, one that included PCA and one that did not. They did not require an imputer because there was no missing data, but I added one anyway—should perform as a noop, but can provide some resiliency in case the data changes. I used the Standard Scaler which normalizes to zero mean and one standard deviation as mentioned earlier. The model was sklearn's SVC.

Aging:

Similarly to above, I had two pipelines, one with PCA and one without. The mean imputer did have values it needed to fill in and used the mean strategy to do that. Standard Scaler was used and then sklearn's SVR for the model.

Modeling & hyperparameters: which kernel/params worked best for classification vs regression, and why.

Explain SVM intuition: margin vs. slack (C), kernels (linear / polynomial / RBF), and why scaling matters.

Margin vs Slack:
In real world data, there will most likely not be a clean hyperplane/line we can draw that will perfectly separate the classes of data points from one another. Therefore, we would intuitively want more slack (low C value that would only give a small penalty for errors on points near the separation line) so that we could have wider margin between the classes. If we have a very narrow margin (and high C value) we will most likely overfit with jagged edges that go around specific points. For this dataset, there is a big risk of overfitting due to the feature to sample ratio so we might see a lower C value to try and correct this.

Kernels:
Linear kernels will separate data without projecting into a higher dimension or having a higher degree polynomial instead of a line. It's good if the data is linearly separable.

Polynomial kernels: Good for data that is separable but not with a straight line. You can go up to arbitrarily high degree polynomials to fit the data but you risk overfitting as the degree increases.

RBF: Good if you aren't able to separate data with lines or polynomials. Projects the data into higher dimensions so that it can separate them there. Should be better for noisier data.

Heart:
For the heart SVC I tested the following kernels/params:

```
kernels = ["linear", "poly", "rbf"]
C = [.1, 1, 10]
gamma = ["scale", "auto", .1]
degree = [2, 3, 4]
```

The best kernel/param combinations here were the degree 3, C=10 polynomial kernel and the .1 gamma, C=10 RBF kernel. The poly degree 4 kernel most likely did worse due to slight overfitting but still was over 97 accuracy and the other high C rbf kernels were also close in performance.

	kernel	gamma	C	degree	accuracy	precision	recall	f1
34	poly	0.1	10.0	3.0	0.990244	0.990414	0.990244	0.990233
44	rbf	0.1	10.0	NaN	0.990244	0.990414	0.990244	0.990233
35	poly	0.1	10.0	4.0	0.975610	0.975680	0.975610	0.975621
16	poly	scale	10.0	3.0	0.975610	0.976643	0.975610	0.975533
25	poly	auto	10.0	3.0	0.975610	0.976643	0.975610	0.975533
38	rbf	scale	10.0	NaN	0.975610	0.976643	0.975610	0.975533
41	rbf	auto	10.0	NaN	0.975610	0.976643	0.975610	0.975533
17	poly	scale	10.0	4.0	0.970732	0.970732	0.970732	0.970732
26	poly	auto	10.0	4.0	0.970732	0.970732	0.970732	0.970732
32	poly	0.1	1.0	4.0	0.960976	0.962357	0.960976	0.960823
31	poly	0.1	1.0	3.0	0.956098	0.957972	0.956098	0.955889
22	poly	auto	1.0	3.0	0.941463	0.943759	0.941463	0.941134

Aging:

For the aging SVR I tested the following kernels/params:

```
kernels = ["linear", "poly", "rbf"]
C = [.1, 1, 10]
gamma = ["scale", "auto", .1]
degree = [2, 3, 4]
```

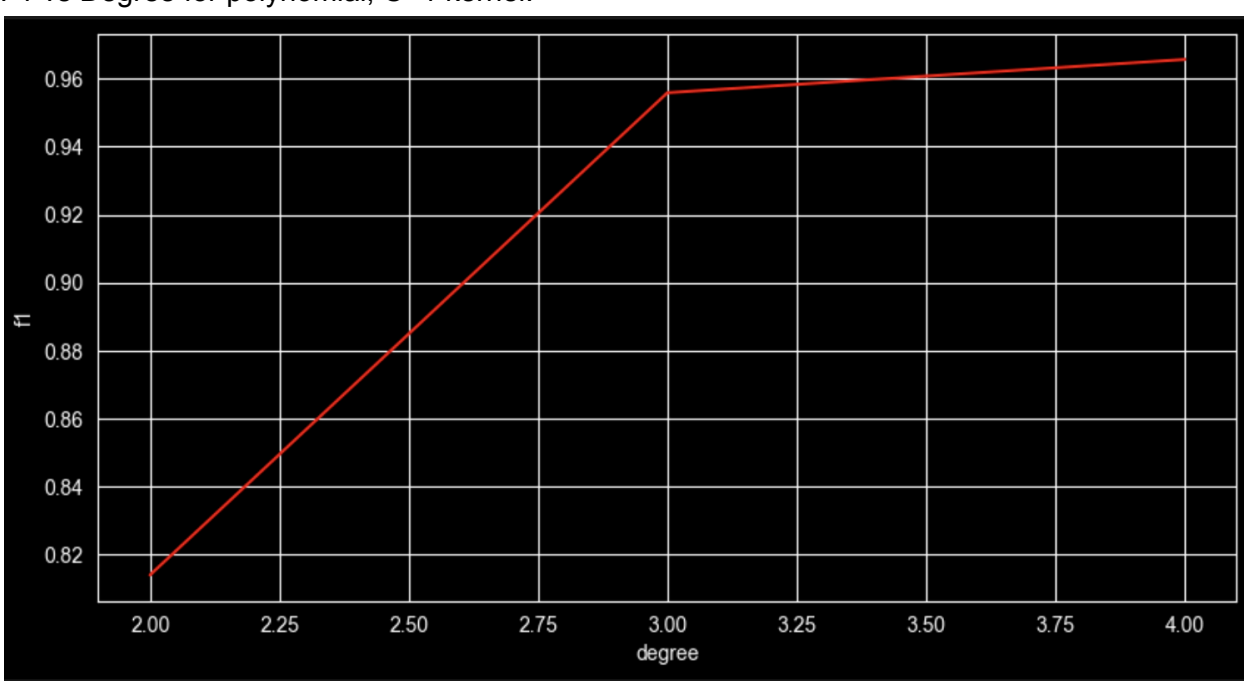
Best param search results:

8	linear	0.1	10.0	NaN	4.286433	5.244763	0.074891
38	rbf	scale	10.0	NaN	4.503829	5.353782	0.036032
41	rbf	auto	10.0	NaN	4.503829	5.353782	0.036032
17	poly	scale	10.0	4.0	4.088819	5.487805	-0.012835
26	poly	auto	10.0	4.0	4.088819	5.487805	-0.012835
16	poly	scale	10.0	3.0	4.203789	5.489951	-0.013627
25	poly	auto	10.0	3.0	4.203789	5.489951	-0.013627
37	rbf	scale	1.0	NaN	3.661978	5.540316	-0.032310
40	rbf	auto	1.0	NaN	3.661978	5.540316	-0.032310

Linear had the best results, which leads us to believe that the other models were overfitting and the only combinations better than just guessing were the scale and auto RBF kernels with a C of 10. The RBF C is a bit counterintuitive since a higher C might cause more overfitting but this could be because of the degree being so small since scale and auto degree are inversely proportional to the amount of features in the dataset. The higher degree polynomial also did better, which makes sense since the data is trivially separable.

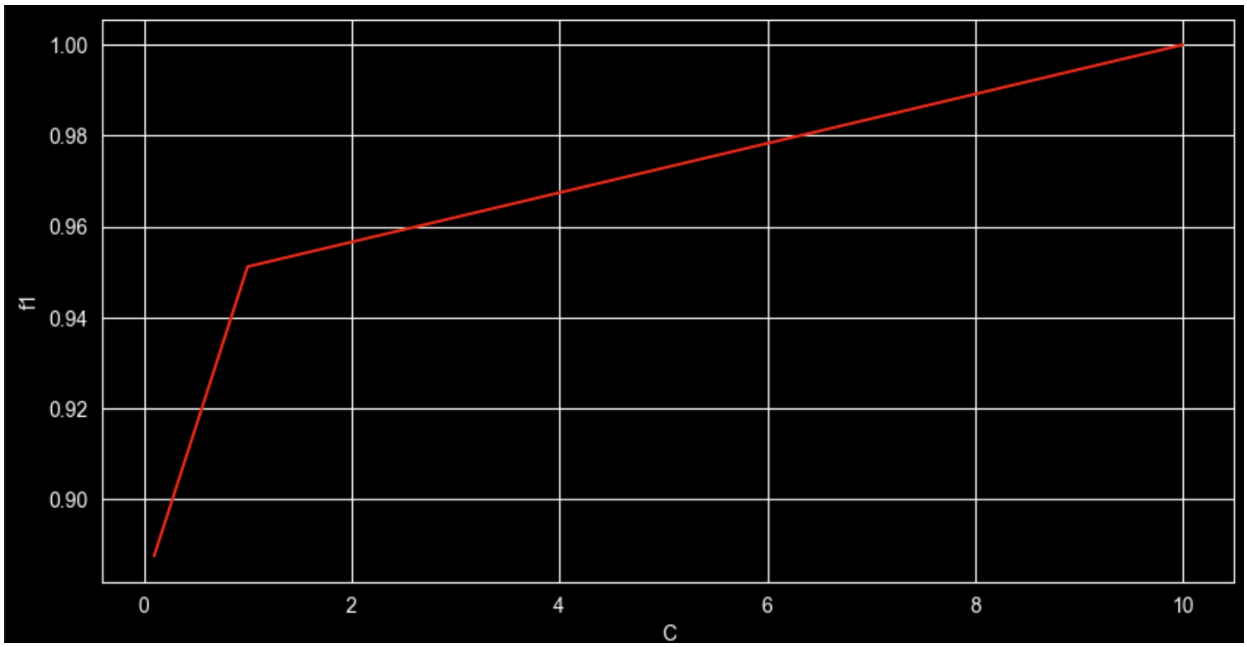
Metrics & visuals: include key scores + confusion matrix/ROC/PR (classification) and parity/residuals (regression)

Heart:
F1 vs Degree for polynomial, C=1 kernel.



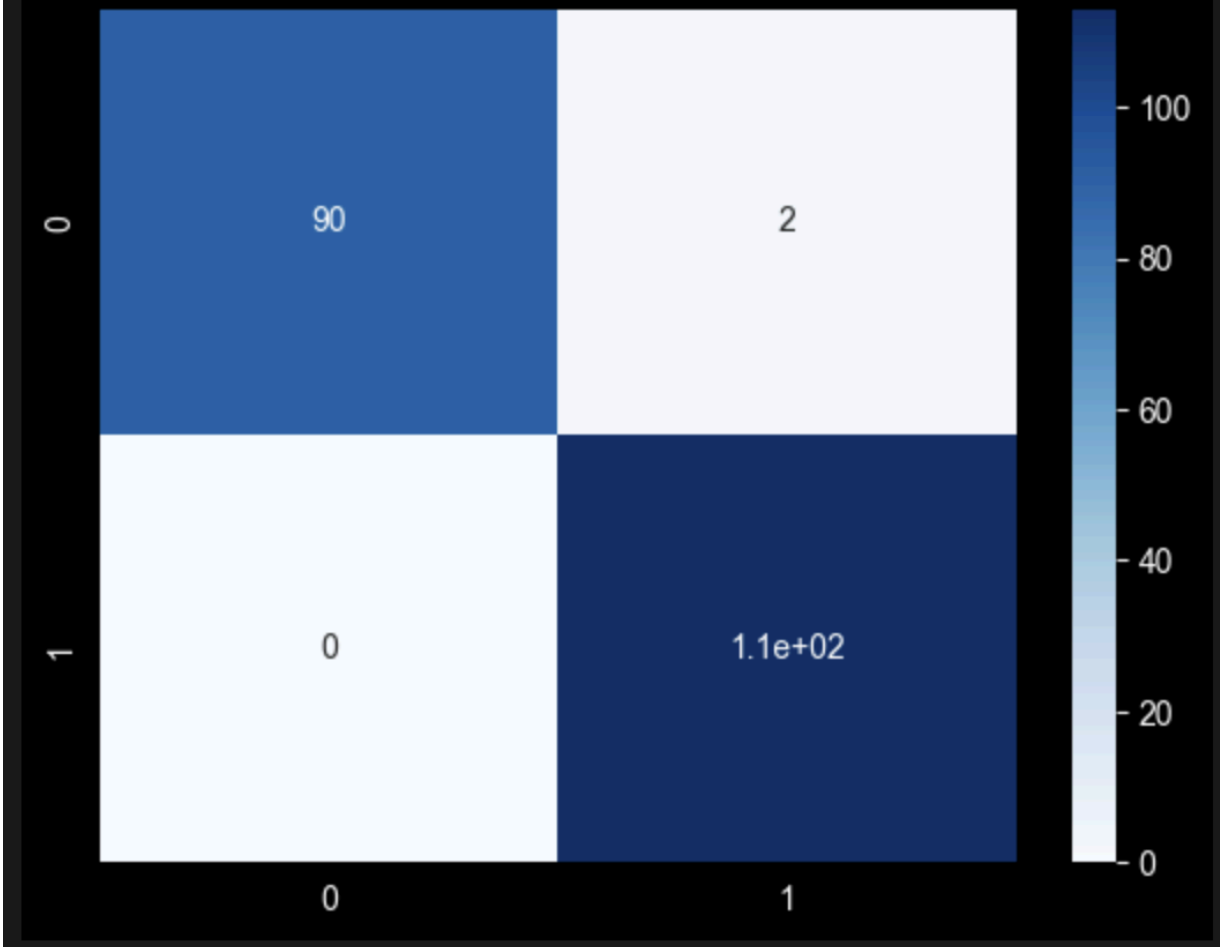
We can see here performance increase dramatically between 2 and 3 degrees and continue to increase at a slower rate between 3 and 4.

F1 vs C for RBF, gamma = .1

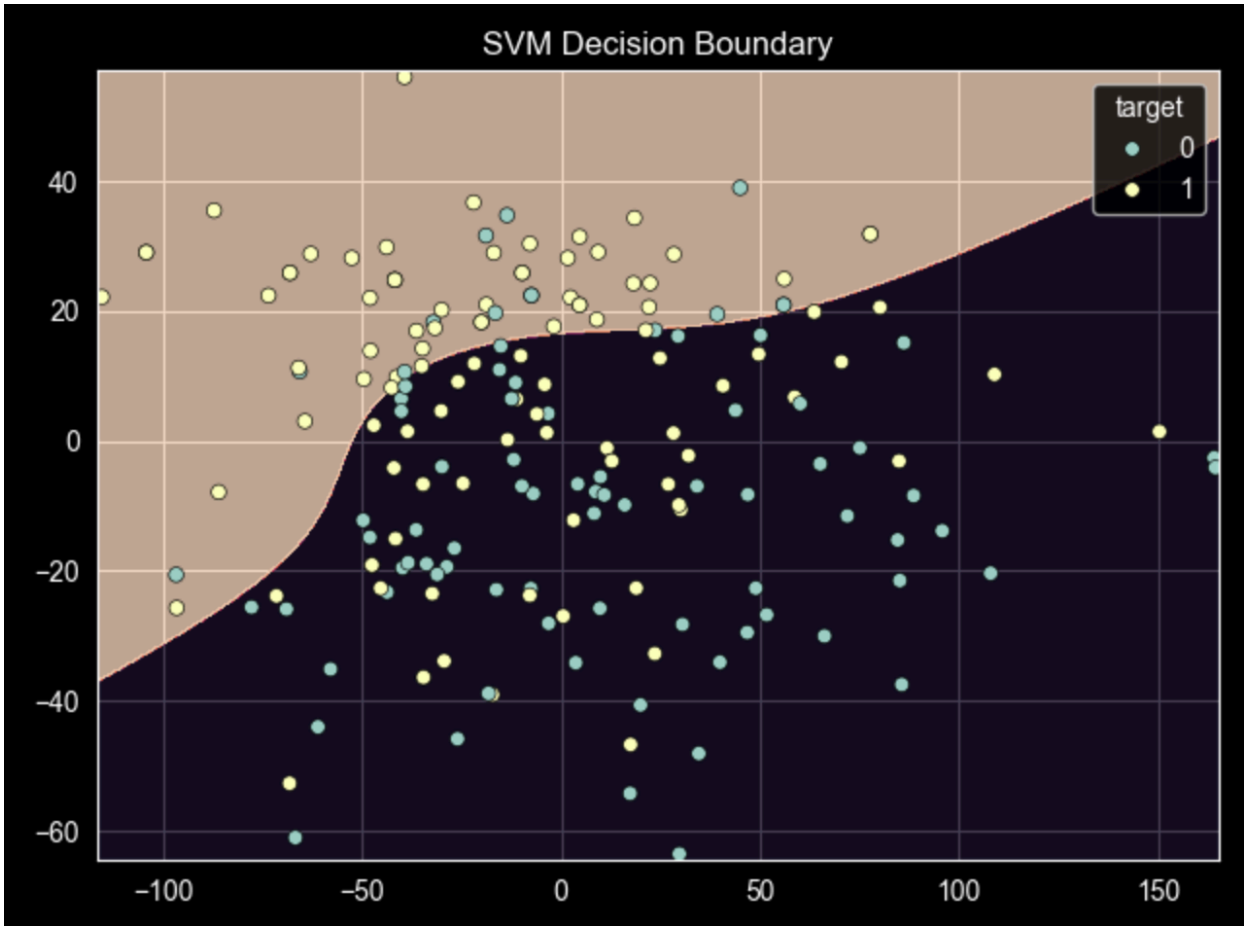


We can see that a higher C value increases F1 score substantially. A higher penalty allows stricter boundaries, which could be beneficial in such a small dataset.

Here is the confusion matrix for the best performing heart model. It is near perfect and an almost 1.0 ROC-AUC score.

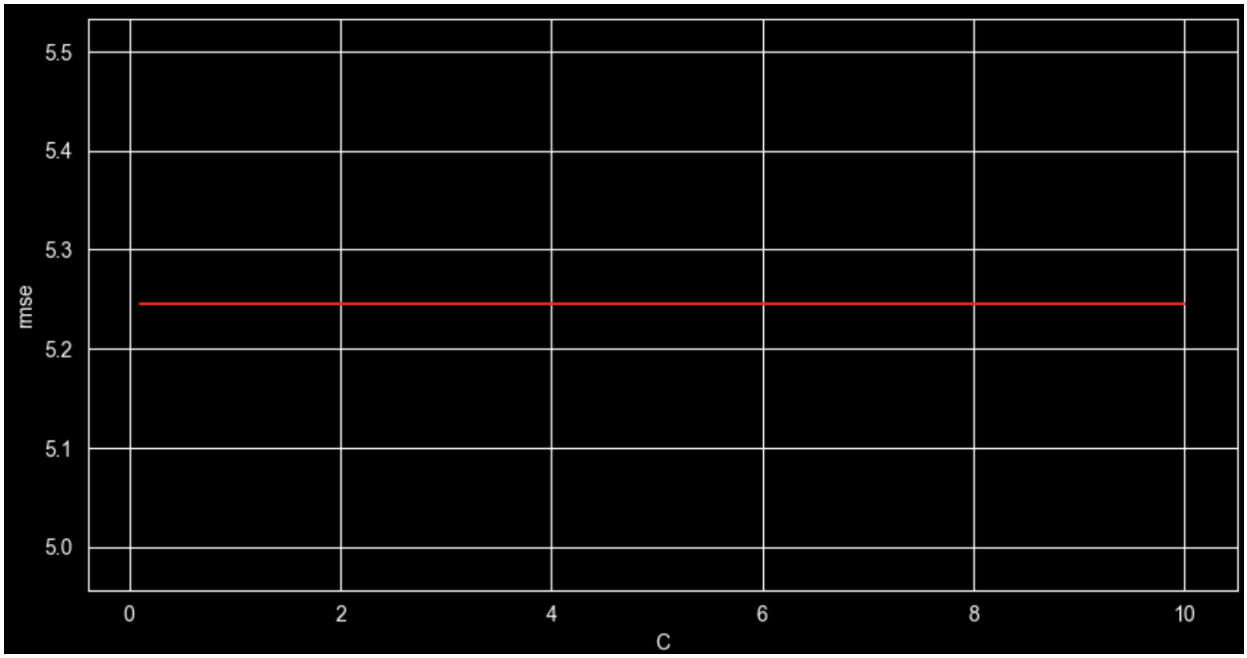



```
Accuracy:  0.9902439024390244
Precision:  0.9904135737009544
ROC AUC:  0.9998076183147364
```

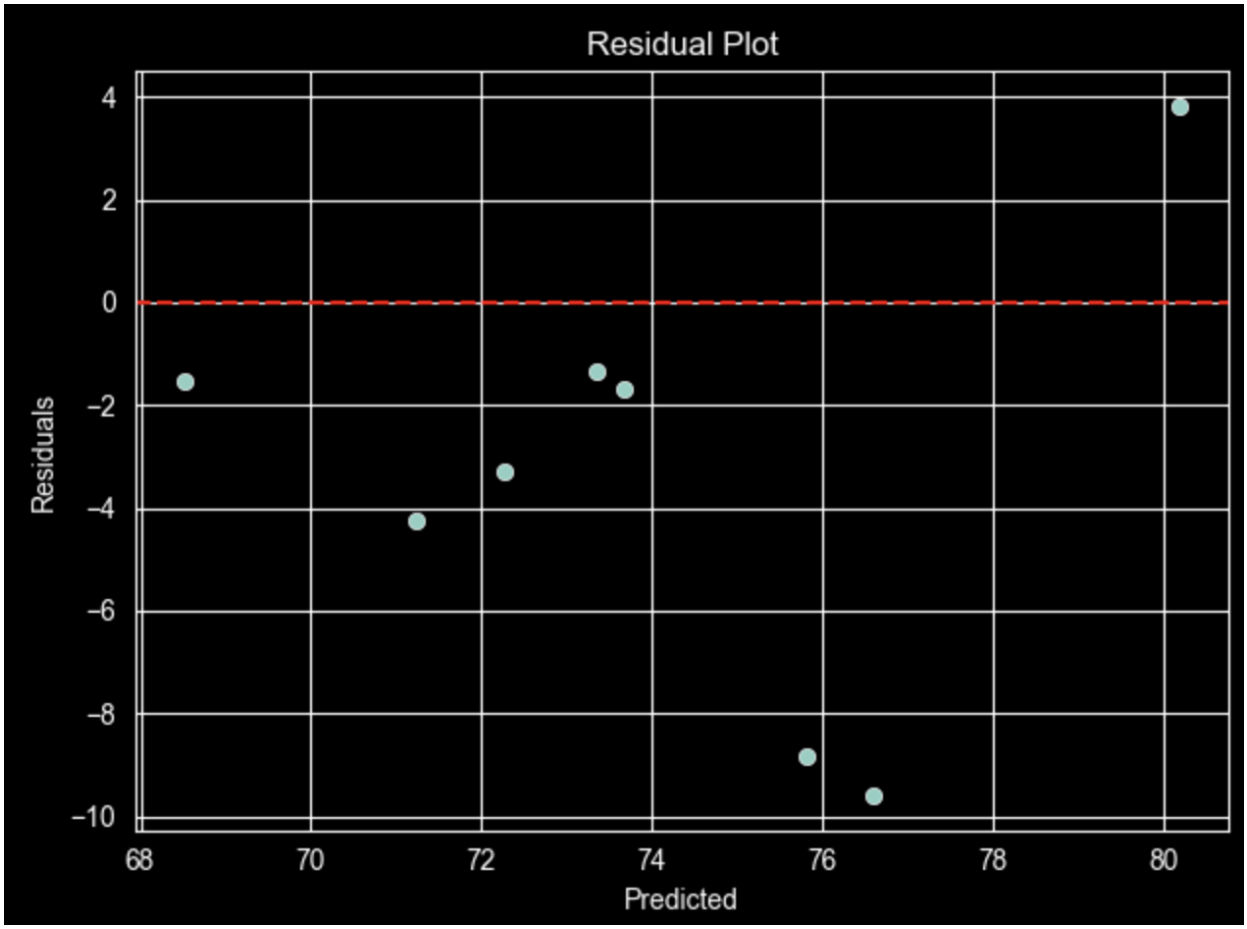


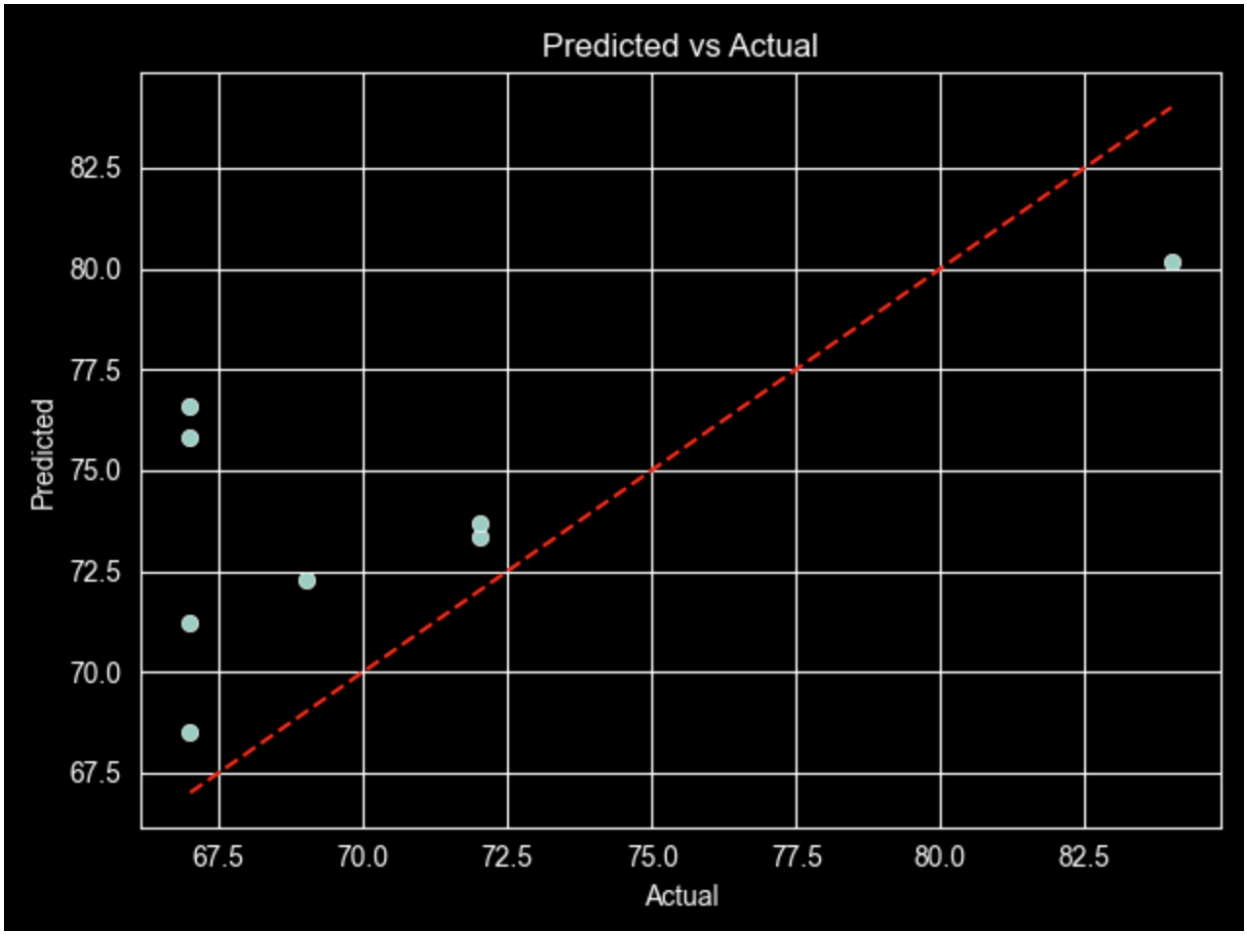
Here is the SVM decision boundary for PCA=2 since we cannot visualize in higher dimensions than that here.

Aging:
RMSE vs C for linear kernel.



This felt a bit unintuitive at first but then thinking about why the linear kernel did best, high degree of freedom when linearly separating the data, it would make sense why the penalty would not end up taking any effect.


Residual Plot
Residuals
Predicted
4
2
0
−2
−4
−6
−8
−10
68
70
72
74
76
78
80



Here we have our residual and parity plots. Visually we can see that residuals are quite far from 0, which is backed up by our poor r2 and high error scores. In our parity plot we can see that the predictions are quite a bit off from what we'd expect but I expected them to be even worse given the metrics we had.

Sensitivity:

We have gone over this earlier, but scaling is necessary for the heart classifier since the values for each feature are on different scales (age vs cholesterol etc…). For the aging dataset (and heart dataset), scaling helps us ensure that all features have the same mean to speed up convergence significantly and the C penalty param also assumes centered data.

With PCA, our results were worse. This shows that even though there are a few features that are most important for the heart dataset, we need all 13 for the optimal model. For aging, the results were similar but reducing the features does not affect the scores as much. This is likely due to the huge amount of features we have.